

DATA STRUCTURES

Previous Year Questions — Complete Solutions

Modules 1, 2 & 3

MODULE 1

Q1. Define data structure. Explain types of data structures.

Definition

A data structure is a systematic way of organizing and storing data in a computer so that it can be accessed and modified efficiently. It defines both the organization of data and the set of operations that can be performed on that data.

Types of Data Structures

Data structures are broadly classified as:

- **Primitive Data Structures**
 - Directly operated upon by machine instructions.
 - Examples: int, float, char, boolean, pointer.
- **Non-Primitive Data Structures**
 - Derived from primitive types; more complex arrangements.
 - Further divided into Linear and Non-Linear types.

Linear Data Structures — elements are arranged sequentially:

- Array — fixed-size collection of elements of the same type, stored in contiguous memory.
- Linked List — nodes connected via pointers; dynamic size.
- Stack — Last-In-First-Out (LIFO) structure.
- Queue — First-In-First-Out (FIFO) structure.

Non-Linear Data Structures — elements are not in sequence:

- Tree — hierarchical structure with nodes and edges (e.g., Binary Tree, BST, AVL Tree).
- Graph — set of vertices connected by edges; models networks.

File Structures — for persistent storage:

- Sequential File, Indexed File, Relative/Direct Access File.

Q2. Explain the concepts of ADT with examples.

Abstract Data Type (ADT)

An Abstract Data Type is a mathematical model for a data type. It defines the type of data stored, the operations that can be performed on it, and the parameters of those operations — without specifying implementation details. The user sees what operations are available, not how they are implemented.

Key Characteristics

- Abstraction — hides implementation details from the user.
- Encapsulation — data and operations are bundled together.
- Reusability — can be implemented differently for different needs.

Example 1: Stack ADT

Data stored: A collection of elements with LIFO ordering.

Operations:

- push(x) — insert element x on top of stack.
- pop() — remove and return top element.
- peek() — return top element without removing.
- isEmpty() — returns true if stack is empty.
- isFull() — returns true if stack is full (for array-based implementation).

Example 2: List ADT

Data stored: An ordered sequence of elements.

Operations:

- insert(pos, x) — insert x at position pos.
- delete(pos) — delete element at position pos.
- find(x) — return position of element x.
- get(pos) — return element at position pos.
- size() — return the number of elements.

Example 3: Queue ADT

- enqueue(x) — insert x at rear.
- dequeue() — remove element from front.
- front() — return front element.
- isEmpty() / isFull() — status checks.

Q3. Define Algorithm and list its properties.

Definition

An algorithm is a finite, well-defined sequence of instructions or steps designed to solve a specific problem or perform a computation. It takes an input, processes it through a series of steps, and produces the correct output.

Properties of an Algorithm (Knuth's Properties)

- Input — An algorithm must have zero or more well-defined inputs.
- Output — It must produce at least one output that is the solution to the problem.
- Definiteness — Each step must be precisely and unambiguously defined.
- Finiteness — The algorithm must terminate after a finite number of steps.

- Effectiveness — Each step must be basic enough to be carried out, in principle, by a person using only pencil and paper.

Additional Desirable Properties

- Correctness — produces correct output for all valid inputs.
- Efficiency — uses minimum time and space.
- Generality — applies to a broad range of inputs.

Q4. Analyze the efficiency of algorithms by explaining and comparing Big O, Big Ω , and Big Θ asymptotic notations.

Why Asymptotic Notation?

As input size grows, constant factors matter less. Asymptotic notations describe how an algorithm's resource usage (time or space) scales with input size n in the limit.

1. Big O Notation — $O(f(n))$ — Upper Bound (Worst Case)

$T(n) = O(f(n))$ means $T(n)$ grows no faster than $cf(n)$ for all $n \geq n_0$ and some constant $c > 0$.

Formal: $\exists c > 0$ and $n_0 > 0$ such that $T(n) \leq c \cdot f(n)$ for all $n \geq n_0$.

Example: $T(n) = 3n^2 + 5n \rightarrow O(n^2)$. We ignore lower-order terms and constants.

Use: Guarantees the algorithm will not exceed this bound — most commonly used in practice.

2. Big Ω Notation — $\Omega(f(n))$ — Lower Bound (Best Case)

$T(n) = \Omega(f(n))$ means $T(n)$ grows at least as fast as $cf(n)$ for sufficiently large n .

Formal: $\exists c > 0$ and $n_0 > 0$ such that $T(n) \geq c \cdot f(n)$ for all $n \geq n_0$.

Example: $T(n) = 3n^2 + 5n \rightarrow \Omega(n^2)$.

Use: Proves that no algorithm can do better than this for the problem.

3. Big Θ Notation — $\Theta(f(n))$ — Tight Bound (Average/Exact)

$T(n) = \Theta(f(n))$ means $T(n)$ grows at exactly the same rate as $f(n)$ — both upper and lower bounds hold.

Formal: $\exists c_1, c_2 > 0$ and $n_0 > 0$ such that $c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$ for all $n \geq n_0$.

Example: $T(n) = 3n^2 + 5n \rightarrow \Theta(n^2)$.

Use: Gives the exact asymptotic growth rate.

Comparison Table

Notation	Type	Meaning	Example
$O(f(n))$	Upper Bound	Worst-case growth rate; algorithm $\leq cf(n)$	$3n^2 + 5n = O(n^2)$
$\Omega(f(n))$	Lower Bound	Best-case growth rate; algorithm $\geq cf(n)$	$3n^2 + 5n = \Omega(n^2)$
$\Theta(f(n))$	Tight Bound	Exact growth rate; both upper & lower hold	$3n^2 + 5n = \Theta(n^2)$

Common Complexity Classes (Fastest to Slowest)

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$

Q5. Explain time and space complexity.

Time Complexity

Time complexity measures the amount of time an algorithm takes to run as a function of input size n . It is expressed in terms of the number of basic operations (comparisons, assignments, etc.) performed.

Types:

- Best Case (Ω) — minimum time for any input of size n . E.g., searching a sorted list where target is the first element: $O(1)$.
- Average Case (Θ) — expected time over all possible inputs of size n .
- Worst Case (O) — maximum time for any input of size n . E.g., linear search where target is last: $O(n)$.

Common Time Complexities

Complexity	Name	Example
$O(1)$	Constant	Array index access
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Linear search
$O(n \log n)$	Linearithmic	Merge sort, Heap sort
$O(n^2)$	Quadratic	Bubble sort, Insertion sort
$O(2^n)$	Exponential	Recursive Fibonacci

Space Complexity

Space complexity measures the total amount of memory space required by an algorithm as a function of input size n , including:

- Instruction space — memory needed to store the compiled code.
- Data space — memory needed for variables and constants.
- Stack space — memory for the function call stack (important for recursion).

Auxiliary Space vs Total Space

- Total space = input space + auxiliary space.
- Auxiliary space is the extra space used by the algorithm beyond the input.
- Example: Merge sort has $O(n)$ auxiliary space (needs temporary array), while in-place bubble sort has $O(1)$ auxiliary space.

Space-Time Tradeoff

Often, using more memory can speed up an algorithm (e.g., memoization/dynamic programming), and vice versa. The goal is to find the right balance for the given problem constraints.

Q6. Differentiate between the following:

(i) Array vs Linked List

Feature	Array	Linked List
Memory allocation	Contiguous (static)	Non-contiguous (dynamic)
Size	Fixed at compile time	Dynamic, grows/shrinks at runtime
Access	$O(1)$ — random access via index	$O(n)$ — sequential traversal
Insertion/Deletion	$O(n)$ — shifting required	$O(1)$ — if pointer available
Memory overhead	No extra memory	Extra memory for pointers
Cache performance	Better (contiguous memory)	Poorer (scattered nodes)
Suitable for	Frequent access, known size	Frequent insert/delete, unknown size

(ii) Stack vs Queue

Feature	Stack	Queue
Principle	LIFO — Last In, First Out	FIFO — First In, First Out
Insertion end	Top (push)	Rear (enqueue)
Deletion end	Top (pop)	Front (dequeue)
Pointers needed	One — top	Two — front and rear
Applications	Function call stack, undo, expression eval	Scheduling, BFS, printer queue

(iii) Top-down vs Bottom-up Approach

Feature	Top-down Approach	Bottom-up Approach
Strategy	Break problem into subproblems (Divide & Conquer)	Solve smallest subproblems first, build up
Implementation	Recursive (with memoization)	Iterative (tabulation)
Memory usage	More (recursive call stack)	Less (iterative table)
Code readability	More intuitive/natural	Less intuitive
Example	Recursive Fibonacci with memoization	DP Fibonacci with loop

(iv) Big-O vs Big-Ω vs Big-Θ

Notation	Bound Type	Formal Definition	Practical Use
$O(f(n))$	Upper (worst-case)	$T(n) \leq c \cdot f(n)$ for $n \geq n_0$	Most common; guarantees max time
$\Omega(f(n))$	Lower (best-case)	$T(n) \geq c \cdot f(n)$ for $n \geq n_0$	Proves no algorithm can beat this
$\Theta(f(n))$	Tight (exact)	$c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$	Exact characterization of growth

MODULE 2

Q1. Define List Abstract Data Type (ADT).

A List ADT is an ordered collection (sequence) of elements where each element has a definite position. The list can grow or shrink dynamically. It supports positional access and various operations to manipulate elements.

Formally: List = (elements: $e_1, e_2, e_3, \dots, e_n$) where $n \geq 0$. When $n = 0$, it is called an empty list.

Key Properties:

- Elements are arranged in a linear order.
 - Duplicate elements may be allowed.
 - Each element (except the first) has a unique predecessor, and each element (except the last) has a unique successor.
-

Q2. List the basic operations performed on List ADT.

- `insert(pos, x)` — Insert element x at position pos in the list.
 - `delete(pos)` — Delete the element at position pos .
 - `find(x)` — Return the position of element x ; return -1 if not found.
 - `get(pos)` — Return the element at position pos (also called retrieve).
 - `update(pos, x)` — Replace element at position pos with x .
 - `size()` / `length()` — Return the total number of elements in the list.
 - `isEmpty()` — Return true if the list has no elements.
 - `clear()` — Remove all elements from the list.
 - `print()` — Display all elements.
-

Q3. Describe Array-based implementation of List with a neat diagram.

Concept

In array-based implementation, the list elements are stored in a contiguous block of memory (array). An integer variable (size or last) keeps track of the current number of elements.

Structure

```
int data[MAX_SIZE];    // Stores list elements
```

```
int size; // Current number of elements
```

Operations and Their Complexity

Operation	Method	Time Complexity
insert(pos, x)	Shift elements right from pos to end, then place x at pos	$O(n)$
delete(pos)	Shift elements left from pos+1 to end to fill gap	$O(n)$
get(pos)	Direct array access: data[pos]	$O(1)$
find(x)	Linear scan of array	$O(n)$
size()	Return value of size variable	$O(1)$

Array Representation Diagram

Before inserting 25 at position 2 (0-indexed) in list [10, 20, 30, 40]:

```
Index:  [ 0 | 1 | 2 | 3 | 4 ]
Data:   [ 10 | 20 | 30 | 40 | _ ] (size = 4)
```

Step: Shift data[2..3] one position right, then data[2] = 25:

```
Data:   [ 10 | 20 | 25 | 30 | 40 ] (size = 5)
```

Advantages

- $O(1)$ random access by index.
- Simple implementation; good cache performance.

Disadvantages

- Fixed size — overflow if more elements than MAX_SIZE.
- $O(n)$ insertion and deletion due to shifting.
- Memory waste if list is much smaller than allocated array.

MODULE 3

Q1. Define Stack and list its basic operations.

Definition

A Stack is a linear data structure that follows the Last In, First Out (LIFO) principle. The element inserted last is the first one to be removed. All insertions and deletions occur at one end called the top of the stack.

A stack can be implemented using an array or a linked list.

Basic Operations

- push(x) — Insert element x onto the top of the stack. If the stack is full, an overflow error occurs.
- pop() — Remove and return the top element. If the stack is empty, an underflow error occurs.

- `peek()` / `top()` — Return the top element without removing it.
 - `isEmpty()` — Returns true if the stack contains no elements (`top == -1` for array implementation).
 - `isFull()` — Returns true if the stack has reached its maximum capacity.
 - `size()` — Returns the current number of elements in the stack.
-

Q2. Define Queue. Explain different types of queue.

Definition

A Queue is a linear data structure that follows the First In, First Out (FIFO) principle. Elements are inserted at the rear end and removed from the front end. It models real-world queues like waiting lines.

Types of Queue

- **Simple / Linear Queue**
 - Basic FIFO queue. Elements inserted at rear, removed from front.
 - Problem: Once front reaches end of array, earlier positions cannot be reused — causes false overflow.
 - **Circular Queue**
 - Overcomes the false overflow problem of linear queue.
 - The rear wraps around to index 0 when it reaches the end: $\text{rear} = (\text{rear} + 1) \% \text{MAX_SIZE}$.
 - Efficiently utilizes all array slots.
 - **Double-Ended Queue (Deque)**
 - Insertions and deletions are allowed at both ends (front and rear).
 - Input-restricted deque: deletion at both ends, insertion at one end only.
 - Output-restricted deque: insertion at both ends, deletion at one end only.
 - **Priority Queue**
 - Each element has an associated priority value.
 - Elements are dequeued in order of their priority, not their insertion order.
 - Implemented using heaps for $O(\log n)$ operations.
 - **Circular Deque**
 - Combination of circular queue and deque — insert/delete at both ends with wraparound.
-

Q3. List applications of Stack.

- Expression Evaluation — Evaluating postfix (Reverse Polish Notation) and prefix expressions.
- Expression Conversion — Converting infix to postfix/prefix using operator precedence rules.
- Balancing Parentheses / Brackets — Checking if expressions have balanced `{}`, `()`, `[]`.
- Function Call Stack — The OS uses a stack to manage function calls, local variables, and return addresses (activation records).
- Undo/Redo Operations — Text editors and software maintain stacks for undo (and redo) history.
- Backtracking Algorithms — Maze solving, N-Queens problem, DFS graph traversal.
- Browser History — Back/forward navigation uses stacks.
- String/Number Reversal — Push all characters/digits, then pop to get reversed output.
- Syntax Parsing — Compilers use stacks to parse grammar and syntax trees.

- Tower of Hanoi — Classic recursive problem solved using stack logic.

Q4. Apply the concept of ADT to explain Stack operations (Push, Pop, Peek, isEmpty, isFull).

Stack ADT — Abstract View

Data: A collection of elements with LIFO ordering. An integer 'top' tracks the topmost element.

1. push(x) — Insert element

```
Algorithm push(x):  
    if isFull():  
        print 'Stack Overflow'  
        return  
    top = top + 1  
    stack[top] = x
```

Time complexity: $O(1)$

2. pop() — Remove top element

```
Algorithm pop():  
    if isEmpty():  
        print 'Stack Underflow'  
        return  
    x = stack[top]  
    top = top - 1  
    return x
```

Time complexity: $O(1)$

3. peek() — View top element

```
Algorithm peek():  
    if isEmpty():  
        print 'Stack is empty'  
        return  
    return stack[top]
```

Time complexity: $O(1)$

4. isEmpty()

```
Algorithm isEmpty():  
    return (top == -1)
```

5. isFull()

```
Algorithm isFull():  
    return (top == MAX_SIZE - 1)
```

Q5. Describe Array representation of Stack with a neat diagram.

Array-based Stack Implementation

A stack is represented using a one-dimensional array of size MAX_SIZE with an integer variable 'top' initialized to -1 (indicating empty stack).

Structure

```
int stack[MAX_SIZE];
int top = -1;          // -1 means empty
```

Push Operation Diagram — Push 10, 20, 30:

```
After push(10):  [10 | __ | __ | __ | __]  top = 0
After push(20):  [10 | 20 | __ | __ | __]  top = 1
After push(30):  [10 | 20 | 30 | __ | __]  top = 2 ← TOP
```

Pop Operation Diagram — Pop once:

```
After pop():     [10 | 20 | 30 | __ | __]  top = 1  (30 returned, top
decremented)
```

Key points

- `top == -1` → stack is empty (underflow condition).
- `top == MAX_SIZE - 1` → stack is full (overflow condition).
- All operations (push, pop, peek) are $O(1)$.
- Major limitation: fixed size — cannot grow beyond `MAX_SIZE`.

Q6. Explain Linked List representation of Stack.

Concept

In linked list-based stack, each element is a node containing data and a pointer to the next node. The 'top' is a pointer to the head (first node). The stack grows and shrinks dynamically — no overflow unless memory is exhausted.

Node Structure

```
struct Node {
    int data;
    struct Node* next;
};
Node* top = NULL;  // Initially empty
```

push(x) — Create node, insert at head

```
void push(int x) {
    Node* newNode = new Node();
    newNode->data = x;
    newNode->next = top;
    top = newNode;
}
```

pop() — Remove head node

```
int pop() {
    if (top == NULL) { print 'Underflow'; return; }
    int val = top->data;
    Node* temp = top;
    top = top->next;
    delete temp;
    return val;
}
```

```
}
```

Linked List Stack Diagram — After push(10), push(20), push(30):

```
top → [30 | *] → [20 | *] → [10 | NULL]
```

Advantages over Array

- Dynamic size — no overflow (memory permitting).
- No wasted memory for unused slots.

Disadvantages

- Extra memory for pointer in each node.
- No random access — only top is accessible.

Q7. Apply Stack operations to reverse a given string or number.

Reversing a String using Stack

Algorithm — Reverse "HELLO":

- Step 1: Push each character onto stack: push(H), push(E), push(L), push(L), push(O).
- Step 2: Stack state (top → bottom): O → L → L → E → H
- Step 3: Pop all characters into a new string: O, L, L, E, H → "OLLEH"

```
Input:   H  E  L  L  O
Stack:   O ← L ← L ← E ← H   (top at left)
Output:  O  L  L  E  H
```

Reversing a Number using Stack

Algorithm — Reverse 12345:

- Step 1: Extract each digit ($12345 \% 10 = 5$, then $1234 \% 10 = 4$, ...) and push: push(5), push(4), push(3), push(2), push(1).
- Step 2: Pop and reconstruct the number: $5 \times 10^4 + 4 \times 10^3 + 3 \times 10^2 + 2 \times 10 + 1 = 54321$.

Q8. Convert infix expressions to prefix and postfix.

Operator Precedence and Associativity

Operator	Precedence	Associativity
^ (exponent)	Highest (3)	Right to Left
* / % (mult/div/mod)	Medium (2)	Left to Right
+ - (add/sub)	Low (1)	Left to Right

Conversion Rules Summary

- Infix → Postfix: Scan left to right; output operands immediately; push operators according to precedence; pop on ')'.

- Infix $\rightarrow$ Prefix: Reverse expression, swap (and), apply postfix algorithm, reverse result.

Expression 1: $(A-B)*(C+D)$

Postfix: $AB-CD+^*$

Prefix: $^*-AB+CD$

Expression 2: $(A+B)*C-D/E$

Postfix: $AB+C*DE/-$

Prefix: $-^*+ABC/DE$

Expression 3: $(A+B)/(C+D)-(D^*E)$

Postfix: $AB+CD+ / DE^*-$

Prefix: $- / + AB + CD * DE$

Expression 4: $A-(B/C+(D\%E^*F)/G)^*H$

Step-by-step: Inner expressions first.

$D\%E \rightarrow DE\%$, then $DE\%F \rightarrow DE\%F^*$, then $DE\%F^*/G \rightarrow DE\%F^*G/$

$B/C \rightarrow BC/$, $BC/ + DE\%F^*G/ \rightarrow BC/DE\%F^*G/+$

$(BC/DE\%F^*G/ +)^*H \rightarrow BC/DE\%F^*G/+H^*$

$A - (\text{above}) \rightarrow ABC/DE\%F^*G/+H^*-$

Postfix: $ABC/DE\%F^*G/+H^*-$

Prefix: $-A^*+/BC/DE\%FGH$ (i.e. $-A^*+/BC/^*\%DEFGH$)

Expression 5: $(A+(B^*C))$

Postfix: ABC^*+

Prefix: $+A^*BC$

Expression 6: $(A-B/C)^*(A/K-L)$

Postfix: $ABC/-AK/L-^*$

Prefix: $^*-A/BC-/AKL$

Expression 7: $(A+B)^*C$

Postfix: $AB+C^*$

Prefix: $^*+ABC$

Expression 8: $(f-g)^*((a+b)^*(c-d)/e)$

Postfix: $fg-ab+cd-^*e/^*$

Prefix: $^*-fg/^*+ab-cde$

Detailed Worked Example — $(A+B)*C-D/E \rightarrow$ Postfix

Step	Token	Stack	Output
1	(	(	
2	A	(	A
3	+	(+	A
4	B	(+	AB
5	)		AB+
6	*	*	AB+
7	C	*	AB+C
8	-	-	AB+C*
9	D	-	AB+C*D
10	/	-/	AB+C*D
11	E	-/	AB+C*DE
12	end		AB+C*DE/-

Final Postfix: AB+C*DE/-

— End of Solutions —